



Arena de Combate

Construí tu propio videojuego con Programación Orientada a Objetos. Este apunte tiene **todo el trabajo práctico en papel**: la consigna, la teoría que vas a necesitar, el código paso a paso, los desafíos, la entrega y la rúbrica. Es la versión imprimible de la guía interactiva, y se puede hacer entero desde acá.

[CONSIGNA](#)[TEORÍA DE REPASO](#)[CÓDIGO PASO A PASO](#)[DESAFÍOS](#)[AYUDAS Y ERRORES](#)[RÚBRICA](#)

Qué hay adentro

- 01 La misión**
Qué hay que construir, cómo se trabaja y el método que se repite en todas las etapas.
- 02 Repaso: la teoría que vas a usar**
Las diez ideas de las clases 1 y 2, en fichas, para tener al lado mientras programás.
- 03 Etapas 1 y 2 · Pensar y diseñar**
Encontrar los objetos y decidir qué va en cada clase. Sin escribir código.
- 04 Etapa 3 · La clase Personaje**
La primera clase: `__init__`, `self`, `return` y el `if` que la protege.
- 05 Etapa 4 · El objeto Arma**
Una clase que no es un personaje, y por qué su método devuelve en vez de mostrar.
- 06 Etapa 5 · El jugador tiene un arma**
Herencia para lo que ES, composición para lo que TIENE. Y `super()`.
- 07 Etapas 6 y 7 · Enemigos y varios niveles**
La segunda clase hija, y qué pasa cuando el hijo tiene a su vez un hijo.
- 08 Etapa 8 · Polimorfismo**
Misma orden, distinta respuesta. Y un `for` sin un solo `if`.
- 09 Etapa 9 · La pelea**
El `while` que hace chocar todo lo que construiste.
- 10 Etapas 10 y 11 · Niveles y menú**
Experiencia, subir de nivel, la poción y la forma de jugarlo.
- 11 El programa completo**
Todo junto, para comparar con el tuyo. Leelo, no lo copies de memoria.
- 12 Desafíos de ampliación**
Cinco niveles, de fácil a difícil, y qué concepto practica cada uno.
- 13 Cuando algo no anda**
Los errores que vas a cometer: síntoma, causa y cómo salir.
- 14 Entrega y evaluación**
Qué hay que entregar, y con qué criterios se corrige.
- 15 Autoevaluación**
Ejercicios de repaso para comprobar que entendiste, no que copiaste.
- 16 Soluciones**
Al final a propósito: primero intentá vos.
- ★ Machete**
La última hoja, para arrancar y llevar: todo el TP en una carilla.





La misión

Bienvenido a la Arena de Combate. Tu misión es construir un videojuego usando Programación Orientada a Objetos.

Nadie te va a dar el juego terminado. Lo vas a ir construyendo vos, etapa por etapa, y en cada una el juego va a poder hacer **una cosa más** que antes. Al principio tu personaje solo va a existir. Después va a poder recibir daño. Después va a tener un arma. Después va a aparecer un enemigo. Y en algún momento vas a tener una pelea por turnos funcionando en tu computadora.

La parte importante no es el juego. Es que al final vas a poder explicar por qué lo hiciste así, y eso te va a servir para cualquier problema, no solo para este.

EL OBJETIVO REAL

Que al terminar puedas explicar, con tus palabras y señalando tu propio código, qué es una clase, qué es un objeto, qué son los atributos y los métodos, cómo funciona la herencia, para qué sirve `super()`, qué diferencia hay entre `print()` y `return`, qué es la composición, qué es el polimorfismo y cómo varios objetos colaboran entre ellos.

Cómo se trabaja

SOLO O EN PAREJA

Individual o en **parejas de máximo dos personas**. Si son dos, los dos tienen que poder explicar todo el código: no sirve repartirse "vos las clases, yo el menú".

PYTHON PURO

Sin librerías raras y sin cosas que no vimos. Todo lo que necesitás ya lo sabés: variables, `if`, `while`, funciones, listas y clases.

UN SOLO ARCHIVO

Abrí un archivo nuevo, ponelo `juego.py` y tenelo al lado mientras avanzás. Cada etapa le agrega algo. **No arranques de cero en cada etapa.**

PAPEL PRIMERO

Varias etapas empiezan sin computadora. No es para hacerte perder tiempo: es para que el código salga solo cuando llegues al teclado.

El método, que no cambia nunca

Este es el camino que se repite en **todas** las etapas, y el que vas a usar el día que te den un problema que no tenga nada que ver con videojuegos.

Paso 1 Pensar el problema ¿De qué se trata? ¿Qué tiene que poder hacer el programa?	Paso 2 Identificar los objetos ¿Qué cosas hay acá? Cada una va a ser una clase.	Paso 3 Atributos y métodos ¿Qué información tiene cada uno? ¿Qué puede hacer?	Paso 4 Pensar las relaciones ¿Cuál es un otro? ¿Cuál tiene un otro?
Paso 5 Diseñar las clases Dibujarlas con cajas y flechas, todavía sin Python.	Paso 6 Escribir el código Recién acá. Cuando ya sabés qué querés, escribirlo es lo fácil.	Paso 7 Probar y detectar errores Ejecutar, ver qué se rompe y entender por qué.	Paso 8 Mejorar Arreglarlo y volver a probar. Eso no es fracasar: eso es programar.

⚠ MIRÁ DÓNDE ESTÁ EL CÓDIGO

En el **sexto** lugar. Cinco pasos antes de tocar el teclado. **No memorices: razoná.** Si arrancás por el código, te vas a trabar — y eso ya lo vimos en las dos clases.

Las 13 etapas de un vistazo

ETAPA 1 Pensar antes de programar Encontrar los objetos de la escena.	ETAPA 2 Diseñar las clases La tabla de atributos y métodos.	ETAPA 3 Clase Personaje La clase de arriba, la que heredan todos.
ETAPA 4 Objeto Arma Una clase que no es un personaje.	ETAPA 5 El jugador tiene un arma Herencia + composición + <code>super()</code> .	ETAPA 6 Crear enemigos La segunda clase hija.
ETAPA 7 Herencia de varios niveles Personaje → Jugador → Guerrero.	ETAPA 8 Polimorfismo Misma orden, distinta respuesta.	ETAPA 9 Construir la pelea El <code>while</code> con todo adentro.
ETAPA 10 Experiencia y niveles Información y comportamiento nuevos.	ETAPA 11 Menú interactivo La forma de usar lo que construiste.	ETAPA 12 Desafíos de ampliación Cinco niveles, de fácil a difícil.

LA VERSIÓN INTERACTIVA

Este apunte es el mismo trabajo práctico que la guía web, que además corrige las actividades sola y tiene simuladores para probar el combate sin instalar nada: pedroibanez97.github.io/P.O.O/tp-arena-de-combate.html
Podés usar los dos: la guía para trabajar y este apunte para estudiar, subrayar y tener a mano.

02

LA AYUDA QUE VAS A TENER AL LADO

Repaso: la teoría que vas a usar

Nada de lo que viene es nuevo: es todo de las clases 1 y 2. Está acá para que no tengas que ir a buscarlo mientras programás.

No hace falta que te lo aprendas de memoria antes de empezar. Arrancá el trabajo práctico y volvé a esta sección cada vez que te trabes. **Esa es la forma en que esto se aprende de verdad.**

Antes que nada: las dos preguntas

PREGUNTA 1

¿Qué información tiene?

nombre

vida

nivel

Lo que el objeto guarda adentro. Son **sustantivos**. En Python los llamamos **atributos**.

PREGUNTA 2

¿Qué puede hacer?

atacar()

curarse()

esta_vivo()

Las acciones que sabe realizar. Son **verbos** y llevan `()`. En Python los llamamos **métodos**.

SI DUDÁS

Decilo en voz alta con las dos frases: "el personaje tiene..." y "el personaje puede...". Una de las dos va a sonar mal. "El personaje tiene atacar" suena horrible: entonces `atacar()` es un método.

Las diez fichas

FICHA 1 · ¿DE DÓNDE SALEN LOS OBJETOS?

Clase y objeto

La clase es **el molde**: describe qué van a tener y qué van a poder hacer todos, pero no existe. El objeto es **uno concreto**, con valores propios. Una clase, todos los objetos que quieras.

```
class Arma:           # el molde
    ...

espada = Arma("Espada", 25) # un objeto
arco = Arma("Arco", 15)    # otro
```

La clase describe. El objeto existe. Cambiarle el daño a la espada no toca al arco: son independientes.

FICHA 2 · LO QUE TIENE Y LO QUE HACE

Atributos y métodos

Los atributos son los datos que el objeto guarda; los métodos, las funciones que viven adentro del objeto y trabajan sobre esos datos.

```
espada.nombre      # atributo: un dato
espada.usar()      # método: una acción
```

Los paréntesis son la pista: **sin paréntesis lo lees o lo cambiás; con paréntesis se ejecuta.**

FICHA 3 · EL MOMENTO EN QUE NACE

`__init__`

Es un método especial que **se ejecuta solo, una sola vez**: justo cuando escribís `Arma("Espada", 25)`. Ahí el objeto recibe sus datos y queda armado. También se lo llama **constructor**.

```
class Arma:
    def __init__(self, nombre, danio):
        self.nombre = nombre
        self.danio = danio
```

Los valores que van entre paréntesis al crear el objeto caen en los parámetros de `__init__`.

FICHA 4 · EL OBJETO HABLANDO DE SÍ MISMO

`self`

Quiere decir "yo". `self.vida` es "mi vida": el objeto refiriéndose a sus propios datos, para no confundirlos con los de otro objeto.

```
def recibir_danio(self, cantidad):
    self.vida = self.vida - cantidad
```

El error número uno: olvidarte el `self.` al guardar un atributo. Si escribís `vida = vida`, el dato no se guarda en el objeto y después "desaparece".

FICHA 5 · SI PODÉS DECIR "ES UN"

Herencia

Escribimos **una sola vez** lo que comparten y cada hijo agrega solo lo suyo. Es **una sola cosa** que recibe todo lo del padre.

```
# Jugador ES UN Personaje
class Jugador(Personaje):
    ...
```

El test: "un jugador **es un** personaje". Si la frase suena rara, no era herencia. Y al revés: **si estás copiando y pegando una clase, ahí hay una herencia esperando.**

FICHA 6 · QUE EL PADRE HAGA SU PARTE

`super()`

`super()` quiere decir "la clase de arriba". Traducido al castellano: "Padre, encargate vos de nombre y vida. Yo me ocupo de lo mío."

```
def __init__(self, nombre, vida, arma):
    super().__init__(nombre, vida)
    self.arma = arma
```

Va como **primera línea** del `__init__` del hijo. Así cada dato se escribe en **un solo lugar** del programa.

FICHA 7 · EL HIJO DE UN HIJO



Varios niveles

Cada escalón **suma, no reemplaza**. El guerrero no recibe solo lo del jugador: recibe todo lo que hay más arriba en la cadena, y todo vive en **un solo objeto**.

```
Personaje → Jugador → Guerrero
nombre     arma     fuerza
vida
```

Dos o tres niveles alcanzan casi siempre. Si necesitás ocho, casi seguro no era herencia lo que buscabas.

FICHA 8 · MOSTRAR NO ES ENTREGAR



print() vs return

`print()` manda algo a la pantalla: lo ve una persona y ahí termina. `return` **te entrega** el valor: lo podés guardar, comparar en un `if` o volver a usar.

```
def esta_vivo(self):
    return self.vida > 0

if jugador.esta_vivo():
    print("Seguí jugando")
```

La pregunta que resuelve la duda: ¿lo necesita una persona o lo necesita el programa?

FICHA 9 · SI PODÉS DECIR "TIENE UN"



Composición

Un atributo también puede ser otro objeto. Son **dos objetos distintos**, conectados: el jugador guarda adentro un arma entera, no un texto ni un número.

```
self.arma = arma    # la línea de siempre

print(jugador.arma.nombre)
# "el nombre del arma del jugador"
```

El punto se lee **"de", de derecha a izquierda**. Y el jugador no sabe cuánto daño hace: **le pregunta al arma**. Cada objeto se ocupa de lo suyo.

FICHA 10 · MISMA ORDEN, DISTINTA RESPUESTA



Polimorfismo

Le damos **la misma orden** a objetos distintos y cada uno responde **a su manera**. El código que los usa no cambia nunca, aunque agregues diez enemigos más.

```
for enemigo in [dragon, goblin, lobo]:
    print(enemigo.atacar())
```

¿Cuántos `if` hay en ese `for`? **Cero**. Cada objeto sabe cómo responder.

Y una que aparece sin que la nombremos: encapsulación

✗ SIN CONTROL

```
jugador.vida = -999999
```

Si cualquier parte del programa puede escribir eso, el juego se rompe: personajes con vida negativa, monedas infinitas, trampa pura. **El dato miente**.

✓ CON CONTROL

```
def recibir_danio(self, cantidad):
    self.vida = self.vida - cantidad
    if self.vida < 0:
        self.vida = 0
```

Ese `if` es la única razón por la que la vida no queda en negativo. Y ese control **vive adentro del objeto**: solo el objeto decide cómo cambian sus propios datos.

Ayuda de sintaxis: lo mínimo que vas a escribir

```
class Nombre:                                # definir una clase (con mayúscula)
    def __init__(self, dato1, dato2):        # el constructor
        self.dato1 = dato1                  # guardar un atributo
        self.dato2 = dato2

    def hacer_algo(self):                    # un método que NO devuelve
        self.dato1 = self.dato1 + 1

    def contestar(self):                    # un método que SÍ devuelve
        return self.dato1

class Hija(Nombre):                          # heredar
    def __init__(self, dato1, dato2, extra):
        super().__init__(dato1, dato2)      # el padre hace su parte
        self.extra = extra                  # el hijo agrega lo suyo

objeto = Hija("a", "b", "c")                # crear el objeto
print(objeto.dato1)                         # leer un atributo
objeto.hacer_algo()                         # ejecutar un método
resultado = objeto.contestar()              # guardar lo que devuelve
```

Si te acordás de este esqueleto, ya podés escribir cualquier clase del trabajo práctico. Todo lo demás son `if`, `while` y listas, que ya sabías desde antes de POO.

03

ETAPAS 1 Y 2 · SIN COMPUTADORA

Pensar y diseñar

ETAPA 1 · OBJETIVO Mirar la escena de un juego y encontrar los objetos, sin escribir una sola línea de código.

Te pasan la captura de un juego y te dicen: "**programalo**". En la pantalla hay un héroe llamado Aria (nivel 3, 80 de 100 de vida, 40 puntos de experiencia), un goblin con 33 de 60 de vida, una espada, una poción y el fondo de la arena.

?

¿Qué objetos necesitamos para construir un juego de combate?

ACTIVIDAD 1**Buscá los objetos en la escena**

Marcá todo lo que te parece que merece ser **un objeto** del programa, o sea, algo con su propia clase. Ojo: no todo lo que ves en la pantalla es un objeto.

- | | |
|--------------------------|--|
| ☐ |  El héroe |
| ☐ |  El goblin |
| ☐ |  La espada |
| ☐ |  La poción |
| ☐ |  La barra de vida |
| ☐ |  Los 40 puntos de experiencia |
| ☐ |  El fondo de la arena |

Después de marcar, escribí al lado de cada una *por qué*. Esa explicación vale más que la marca. La solución está en la sección 16.

LA REGLA

Un **objeto** es algo que tiene información y *además* hace algo. Si solo guarda un dato y no hace nada, casi siempre es un **atributo de alguien más**. Que algo se te ocurra no significa que merezca ser una clase.

ACTIVIDAD 2**Atributo o método**

Le hacemos las dos preguntas al héroe. Escribí **A** (atributo) o **M** (método) al lado de cada uno, y probá leyéndolo en voz alta con "el héroe tiene..." / "el héroe puede...".

nombre

atacar()

vida

recibir_danio()

experiencia

subir_nivel()

Un objeto que todavía no viste

Mirá al héroe y al goblin. Los dos tienen **nombre**, los dos tienen **vida**, a los dos les pueden pegar y los dos pueden morir. Si todo eso lo comparten, escribirlo dos veces sería un error: acordate de que **si estás copiando y pegando una clase, ahí hay una herencia esperando**.

Entonces creamos una clase de arriba con lo que comparten. La vamos a llamar **Personaje**, y de ahí van a heredar el jugador y el enemigo.

ETAPA 2 • OBJETIVO

Dejar decidido, en una tabla, qué va a tener y qué va a poder hacer cada clase. Recién después, código.



¿Qué información tiene cada clase y qué puede hacer cada una?

ACTIVIDAD 3

Completá la tabla de diseño

Primero la tuya. No mires ninguna solución todavía: escribí lo que **vos** creés, aunque no estés seguro. Esto se hace en papel y tarda cinco minutos; saltarlo tarda dos horas.

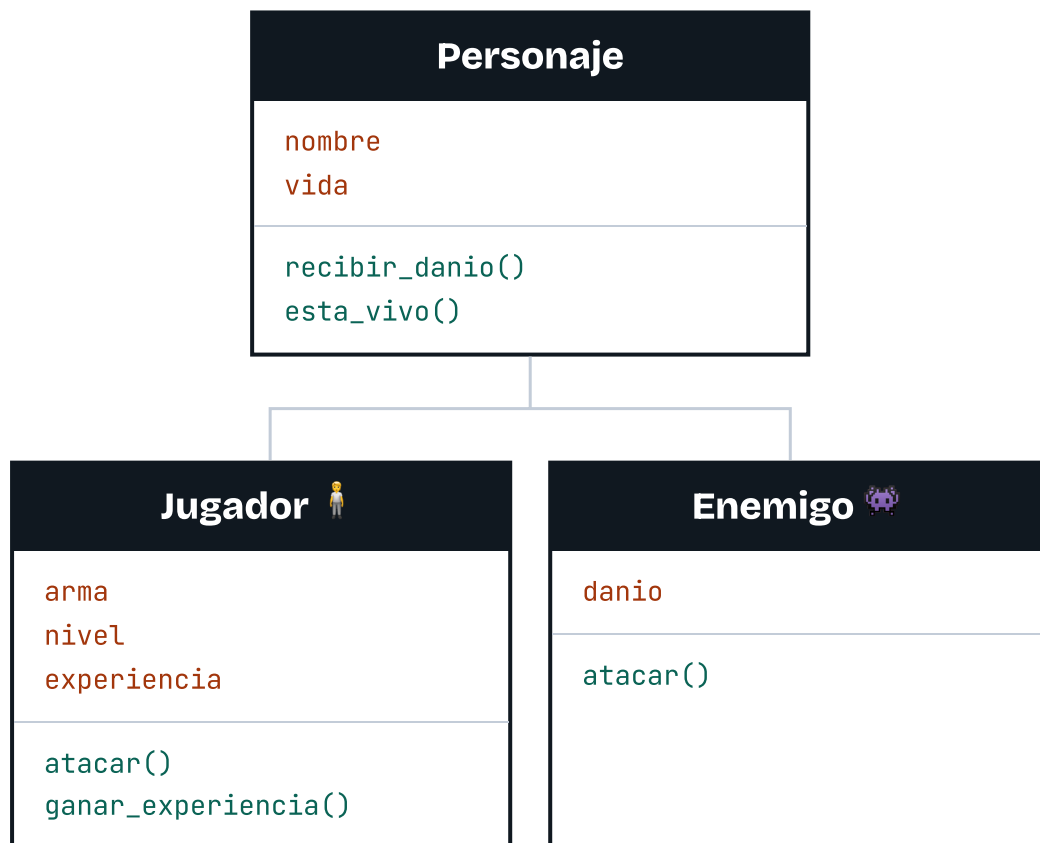
CLASE	ATRIBUTOS • ¿QUÉ TIENE?	MÉTODOS • ¿QUÉ PUEDE HACER?
Personaje la de arriba		
Jugador		
Enemigo		
Arma		

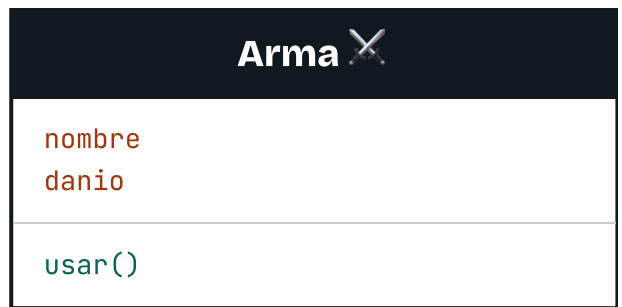
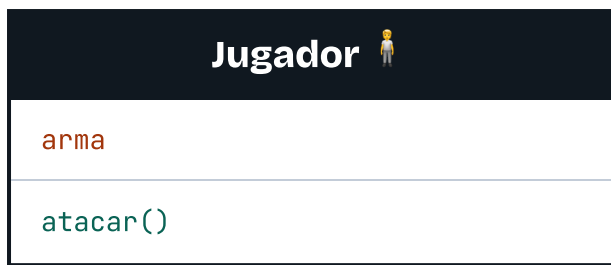
Ojo: en Jugador y en Enemigo **no repitas** `nombre` ni `vida`. Eso ya está en Personaje y lo van a heredar. En la tabla de un hijo va **solo lo que agrega**.

⚠ LA DECISIÓN DIFÍCIL DEL DISEÑO

No es *qué* atributos existen: es **en qué clase va cada uno**. La regla es siempre la misma: **arriba va solo lo que comparten todos**. Si le ponés `experiencia` a Personaje, entonces los goblins, las piedras y los cofres también tienen experiencia. Y el `arma` no va en Personaje: no todos los personajes tienen un arma.

El diseño que vamos a construir





Dos flechas, dos relaciones distintas. La de arriba dice **ES UN** y es herencia. La del costado dice **TIENE UN** y es composición. Copiá este dibujo en tu carpeta: es parte de la entrega final.

⚖️ PUEDE HABER OTRAS RESPUESTAS CORRECTAS

Si vos pusiste `vida_maxima` en Personaje, o un método `mostrar_info()`, o le pusiste `fuerza` al Jugador: **está bien**, siempre que puedas justificarlo. Diseñar es tomar decisiones y casi nunca hay una sola correcta.

04

ETAPA 3 · AHORA SÍ, AL TECLADO

La clase Personaje

OBJETIVO Escribir la primera clase del juego: la de arriba, la que van a heredar todos los demás.

? ¿Por qué conviene empezar por `Personaje` y no por `Jugador`?

Porque los demás van a heredar de ella: **primero tiene que existir el padre**. Y además, poner lo común arriba es justamente lo que evita repetir código.

Paso 1 · los atributos

```
class Personaje:
    def __init__(self, nombre, vida):
        self.nombre = nombre
        self.vida = vida
```

Todavía no existe ningún personaje: solo dijimos **cómo se fabrican**. La clase se escribe con mayúscula, por convención.

Paso 2 · que pueda recibir daño

```
def recibir_danio(self, cantidad):
    self.vida = self.vida - cantidad
```

Un personaje al que no le podés pegar no sirve para una arena. Este método **modifica los datos del propio objeto**: nadie le toca la vida desde afuera.

📝 NOTA SOBRE LOS NOMBRES

En la clase 2 lo escribimos `recibir_daño`, con ñ. Acá lo escribimos `recibir_danio`, sin ñ. **Python acepta las dos**, pero sin ñ te evitás problemas al copiar código, al cambiar de teclado o al compartir el archivo. Es exactamente la misma idea, con otro nombre.

Paso 3 · que pueda contestar si sigue vivo

```
def esta_vivo(self):
    return self.vida > 0
```

El método más importante de todo el juego, aunque parezca el más chiquito: no imprime nada, **devuelve** `True` o `False`.

¿POR QUÉ `ESTA_VIVO()` USA `RETURN`?

Porque **contesta una pregunta** y el programa va a usar esa respuesta adentro de un `if` y del `while` de la pelea. Si hiciera `print`, el `while` sería imposible de escribir.

¿Y POR QUÉ `RECIBIR_DANIO()` NO?

Porque **hace un cambio en el objeto** y listo: modifica la vida y no tiene nada que entregar. Siempre la misma pregunta: **¿lo necesita una persona o el programa?**

PREDICCIÓN

Antes de ejecutar

1. Si la vida es **100** y el personaje recibe **30** de daño, ¿cuánta vida le queda?
2. Si la vida es **0**, ¿qué devuelve `esta_vivo()`?
3. ¿Y si el personaje tiene 100 de vida y recibe **500** de daño?

⚠ EL PROBLEMA ESCONDIDO

La vida queda en **-400**. Y un personaje con vida negativa no existe: **el dato miente**. Esto ya lo arreglaste una vez, en el ejercicio de la mascota de la clase 1.

DESAFÍO

Arreglalo vos

Hacé que la vida **nunca** quede por debajo de cero. Completá los tres huecos:

```
def recibir_danio(self, cantidad):
    self.vida = self.vida - cantidad
    ----- self.vida ___ 0:
        self.vida = ___
```

Pista: primero restás y **después** te fijás si te pasaste. El arreglo tiene que vivir **adentro del objeto**, porque es el objeto el que decide cómo cambian sus propios datos. Eso es **encapsulación**.

EN LA COMPU

Probalo

Escribí la clase completa en `juego.py` y, **antes de ejecutar**, anotá en la carpeta qué esperás que imprima cada línea:

```
aria = Personaje("Aria", 100)
aria.recibir_danio(30)
print(aria.vida)
aria.recibir_danio(500)
print(aria.vida)
print(aria.esta_vivo())
```

ETAPA 4

05

El objeto Arma

OBJETIVO Fabricar un objeto que no es un personaje, y ver por qué su método devuelve en vez de mostrar.



¿Un Arma es un Personaje? ¿Debería heredar de Personaje?

No. Leelo en voz alta: "una espada es un personaje". Suena raro, y cuando la frase suena rara, la respuesta es no. ¿Cuánta vida tiene una espada? ¿Se muere? Es **otra cosa**: una clase aparte, que no hereda de nadie.

```
class Arma:
    def __init__(self, nombre, danio):
        self.nombre = nombre
        self.danio = danio

    def usar(self):
        return self.danio
```

Las dos preguntas de siempre. ¿Qué tiene? Un nombre y un daño. ¿Qué puede hacer? Usarse: entregar cuánto daño hace.



POR QUÉ USAR() DEVUELVE

Porque ese daño **no es para mirarlo**: es para restárselo a la vida del enemigo. Lo necesita el **programa**, no una persona. Con `print` el número se vería en pantalla y se perdería, y nadie podría agarrarlo.



NOTA SOBRE LOS NOMBRES

En la clase 2 este método se llamaba `atacar()`. Acá le decimos `usar()` porque el arma **no ataca sola**: la usa alguien. El nombre lo elegís vos; lo que no se negocia es que **devuelva** el daño.

Del molde a las armas de verdad

```
espada = Arma("Espada", 25)
arco = Arma("Arco", 15)

print(espada.usar())      # ?
print(arco.usar())        # ?
print(espada.nombre)      # ?

resultado = espada.usar()  # el número queda GUARDADO
print(resultado - 5)       # ← puedo seguir haciendo cuentas
```

Un solo molde, dos armas **independientes**. Predecí qué imprime cada línea antes de ejecutarlo. La última es la importante: como `usar()` devuelve, el valor se puede guardar en una variable y seguir usándolo. Con `print`, `resultado` valdría `None`.

ENTRENAMIENTO

¿print() o return?

Para cada método del juego, escribí cuál corresponde y por qué:

1. Un método que calcula el daño de un golpe.
2. Un método que muestra el menú de opciones.
3. Un método que dice si el enemigo sigue vivo.
4. Un método que saluda al empezar la partida.
5. Un método que calcula cuánta experiencia falta para el próximo nivel.

DESAFÍO**Forjá tu propia arma**

Inventá el arma con la que va a pelear **tu** personaje: ponele el nombre que quieras y decidí cuánto daño hace. Te va a acompañar durante todo el trabajo práctico.

Escribila en `juego.py` y probá `print(mi_arma.usar())`.

06**ETAPA 5 · EL CORAZÓN DEL TRABAJO PRÁCTICO****El jugador *tiene* un arma**

OBJETIVO Conectar dos objetos: herencia para lo que **es**, composición para lo que **tiene**.

Tenemos un **Personaje** y tenemos un **Arma**. Están sueltos: ninguno de los dos sabe que el otro existe. Hay que decidir **cómo se relacionan** — y eso se decide *antes* de escribir código, leyendo dos frases en voz alta.

PREGUNTA 1**¿Un Jugador ES un Arma?**

"Un jugador es un arma." Suena horrible. Y cuando la frase suena rara, **la respuesta es no**. Acá **no** va herencia.

PREGUNTA 2**¿Un Jugador TIENE un Arma?**

"Un jugador tiene un arma." Esa sí suena natural. Entonces va **composición**: dos objetos distintos, conectados.

Paso 1 · la herencia y el `super()`

```
class Jugador(Personaje):  
    def __init__(self, nombre, vida):  
        super().__init__(nombre, vida)
```

El `(Personaje)` es todo lo que hace falta para decir "**Jugador ES UN Personaje**": con eso el jugador ya tiene `recibir_danio()` y `esta_vivo()` sin escribirlos. Y `super()` es pedirle al padre que guarde `nombre` y `vida`, para no repetirlos.

PASO 1 Python entra al `__init__` de **Jugador** con los datos que le pasamos.

PASO 2 La línea `super().__init__(nombre, vida)` **sube a Personaje**, que guarda `nombre` y `vida`.

PASO 3 Vuelve a **Jugador**, que guarda lo suyo. Listo: el objeto quedó completo, y al usarlo **no se nota** quién puso cada cosa.

DETECTÁ EL ERROR

Esto explota en la última línea

```
class Jugador(Personaje):
    def __init__(self, nombre, vida):
        self.vida = vida          # se olvidó del super()

aria = Jugador("Aria", 100)
print(aria.nombre)
```

¿Qué error da y por qué? Escribilo con tus palabras antes de mirar la sección 16.

Paso 2 • el arma entra adentro del jugador

```
class Jugador(Personaje):
    def __init__(self, nombre, vida, arma):
        super().__init__(nombre, vida)
        self.arma = arma          # ← ¡acá!

espada = Arma("Espada", 25)
aria = Jugador("Aria", 100, espada)
```

Primero fabricamos el arma, **después** el jugador pasándole esa arma. El objeto `espada` entra adentro del objeto `aria`.

📦 TODA LA NOVEDAD, EN UNA LÍNEA

`self.arma = arma` es la misma línea de siempre: `self.algo = algo`. Lo único distinto es **qué** estamos guardando: no es un texto ni un número, es un **objeto entero**. Un atributo también puede ser otro objeto.

aria

nombre → "Aria"
vida → 100
arma → ♦

APUNTA A

espada

nombre → "Espada"
danio → 25

```
print(aria.nombre)          # Aria
print(aria.arma.nombre)     # Espada
print(aria.arma.usar())     # 25
```

El punto se lee **"de", de derecha a izquierda**: `aria.arma.nombre` es "el nombre **del** arma **de** Aria". Cada punto es un paso más adentro. Si te perdés en una línea larga con muchos puntos, leela así y se ordena sola.

Paso 3 • que ataque: dos objetos colaborando

```
def atacar(self):
    return self.arma.usar()
```

La línea más importante de la etapa.

El jugador *no sabe* cuánto daño hace. Le *pregunta al arma*.

En `return self.arma.usar()` participan **dos objetos**: el jugador le pide el daño a *su* arma y devuelve lo que el arma le contestó. No hay ninguna cuenta en el código del jugador. **Cada objeto se ocupa de lo suyo: eso es que los objetos "hablen entre ellos".**

¿Y PARA QUÉ SIRVE TANTO LÍO? PARA CAMBIAR DE ARMA

```
hacha = Arma("Hacha", 40)
aria.arma = hacha
print(aria.atacar())    # 40
```

¿Qué línea de la clase `Jugador` tuvimos que cambiar? Ninguna. El jugador siempre supo lo mismo: *"tengo un arma y le pido que ataque"*. Qué arma sea, no es su problema. Con 50 armas distintas la clase sigue igual de corta; si hubiéramos usado herencia, cada arma nueva sería un tipo de jugador nuevo. Un desastre.

ENTRENAMIENTO

ES UN o TIENE UN

Para cada par, escribí **ES UN** (herencia) o **TIENE UN** (composición). Leelo en voz alta con las dos frases antes de contestar.

1. Jugador / Personaje
2. Jugador / Arma
3. Auto / Rueda
4. Perro / Animal
5. Celular / Batería
6. JefeFinal / Enemigo
7. Jugador / Inventario
8. Guerrero / Jugador

EN LA COMPU

Desafío de la etapa

1. Escribí la clase `Jugador` completa. Creá tu arma y tu jugador, y mostrá `print(jugador.atacar())`.
2. Mostrá el nombre del arma del jugador en una sola línea.
3. Creá una segunda arma, cambiásela al jugador y volvé a atacar. **Sin tocar la clase `Jugador`**.
4. **La difícil:** hacé que el jugador le pegue a un personaje de verdad, en dos líneas.

07

ETAPAS 6 Y 7

Enemigos y herencia de varios niveles

ETAPA 6 • OBJETIVO Escribir una segunda clase hija y ver que la herencia se escribe una vez y se usa muchas.

Tu jugador pega, pero del otro lado no hay nadie. **Falta el enemigo.** Nuestro goblin pelea a mordiscones: no usa arma, tiene un `danio` propio. (También se podría darle un `Arma` llamada "mordisco" — las dos decisiones son defendibles; lo importante es poder justificarla.)

```
class Enemigo(Personaje):
    def __init__(self, nombre, vida, danio):
        super().__init__(nombre, vida)
        self.danio = danio

    def atacar(self):
        return self.danio
```

Es el **mismo molde** que `Jugador`, cambiando lo que agrega. Si ya lo podés escribir casi sin pensar, es que entendiste el patrón.

QUÉ RECIBE DE PERSONAJE

Los atributos `nombre` y `vida`, y los métodos `recibir_danio()` y `esta_vivo()`. **Ninguno está escrito adentro de `Enemigo`**: los hereda.

QUÉ AGREGA ENEMIGO

El atributo `danio` y el método `atacar()`. Eso es lo único suyo. Lo común (`nombre` y `vida`) está arriba; lo específico, abajo.

```
goblin = Enemigo("Goblin", 60, 12)
esqueleto = Enemigo("Esqueleto", 45, 18)
lobo = Enemigo("Lobo", 70, 10)
```

Con la clase escrita **una sola vez** ya podés poblar toda la arena. Tres objetos independientes: bajarle la vida a uno no toca a los otros.

COMPROBÁ

¿Qué tiene el objeto goblin?

Marcá cuáles de estas líneas funcionan y cuáles dan error, y explicá por qué:

`goblin.nombre`

`goblin.vida`

`goblin.danio`

`goblin.esta_vivo()`

`goblin.recibir_danio(20)`

`goblin.arma`

Y después, en la carpeta: ¿qué atributos de tu enemigo vienen de `Personaje` y cuáles son propios de `Enemigo`?

ETAPA 7 • OBJETIVO

Ver qué pasa cuando el hijo tiene a su vez un hijo, y hasta dónde conviene estirar la cadena.

Tu juego crece: querés que existan **clases de personaje**, como en cualquier RPG. Empezamos por el **Guerrero**, que es un jugador con **fuerza**. Hacé el test de siempre, escalón por escalón: *"un guerrero es un jugador"* ✓, *"un jugador es un personaje"* ✓. Los dos escalones aguantan la frase.



```
class Guerrero(Jugador):
    def __init__(self, nombre, vida, arma, fuerza):
        super().__init__(nombre, vida, arma)
        self.fuerza = fuerza
```

¿Qué hay de nuevo acá? **Nada**. Es exactamente lo mismo de la etapa 5, hecho una vez más. Y fijate: `Guerrero` **nunca** **Menciona** `vida` . Eso es asunto de `Personaje` .

LA IDEA CENTRAL

Cada escalón suma, no reemplaza. `Guerrero` le pide a `Jugador` , y `Jugador` le pide a `Personaje` . Cada clase se ocupa solamente de lo suyo y nadie repite el trabajo de otro. Los cuatro atributos terminan viviendo en **un solo objeto**.

PREDICCIÓN ¿Cuáles funcionan?

```
espada = Arma("Espada", 25)
aria = Guerrero("Aria", 100, espada, 8)

print(aria.nombre)
print(aria.arma.nombre)
print(aria.fuerza)
print(aria.esta_vivo())
print(aria.danio)
```

Escribí qué imprime cada línea. **Una de las cinco da error:** encontrala y explicá por qué.

⚠️ QUE SE PUEDA NO SIGNIFICA QUE CONVENGA

Una cadena de ocho clases hace que, para entender **una**, haya que leer **ocho**. **Dos o tres niveles alcanzan casi siempre**. Si necesitás ocho, casi seguro no era herencia lo que buscabas.

1. Escribí `Guerrero` y creá uno. Imprimí sus cuatro datos.
2. Escribí **otra** clase hija de `Jugador`: un `Mago` que agregue `magia`.
3. En la carpeta: ¿de quién hereda el `Mago` directamente? ¿Y qué atributos termina teniendo?
4. **La difícil**: hacé que el `Guerrero` pegue más fuerte que un jugador común usando su `fuerza`. Escribible su propio `atacar()`. Eso ya es polimorfismo, que es justo lo que viene ahora.

08

ETAPA 8

Polimorfismo: misma orden, distinta respuesta

OBJETIVO Que todos los enemigos entiendan `atacar()`, pero que cada uno conteste a su manera.

Tu arena ya tiene goblins, esqueletos y lobos... y **los tres pegan exactamente igual**. Aburridísimo. Queremos que el dragón sea temible y que el goblin sea un estorbo menor.

x LA IDEA QUE SALE SOLA

```
if enemigo.tipo == "dragon":
    danio = enemigo.danio + 15
elif enemigo.tipo == "goblin":
    danio = enemigo.danio
elif enemigo.tipo == "esqueleto":
    danio = enemigo.danio * 2
# ... y así con cada enemigo nuevo
```

✓ QUE CADA UNO SEPA ATACAR

```
class Dragon(Enemigo):
    def atacar(self):
        return self.danio + 15

class Goblin(Enemigo):
    def atacar(self):
        return self.danio
```

El problema del primero no es que no funcione: **es que crece con cada enemigo nuevo**. Cada vez hay que volver a ese `if` y agregarle un `elif` — y si el mismo `if` está en tres lugares del programa, hay que acordarse de los tres.

Ya sabíamos que un hijo puede **agregar** cosas. Acá usamos la otra mitad de la herencia: un hijo también puede **reemplazar** un método del padre. Ninguno de los dos escribe `__init__`: lo heredan tal cual, porque no agregan ningún atributo nuevo. Solo cambian **cómo responden** a una orden.

```
dragon = Dragon("Dragón", 90, 20)
goblin = Goblin("Goblin", 60, 12)
lobo = Goblin("Lobo", 70, 10)

for enemigo in [dragon, goblin, lobo]:
    print(enemigo.nombre, "ataca con", enemigo.atacar())
```

Predecí qué imprime, antes de ejecutarlo. Y después contesté la pregunta que importa: ¿cuántos `if` hay en ese `for`?

👉 LA RESPUESTA

Cero. El `for` no sabe ni le importa quién es cada uno: **cada objeto sabe cómo responder**. Y si mañana agregás un `Fantasma`, no tocás una sola línea del `for`. El juego solo grita `atacar()`; cada personaje sabe qué significa eso para él.

UNA PREGUNTA PARA PENSAR

¿Hacía falta escribir `atacar()` en `Goblin`? **No**. Si lo borrás, el goblin igual sabe atacar: lo hereda de `Enemigo`, que hace exactamente lo mismo. Lo dejamos escrito solo para poder **comparar** las dos clases una al lado de la otra. En un programa de verdad, un hijo escribe un método **solo cuando lo hace distinto**.

EN LA COMPU

Desafío de la etapa

1. Creá un `Esqueleto(Enemigo)` cuyo `atacar()` devuelva `self.danio * 2`.
2. Creá un `Slime(Enemigo)` que pegue la mitad: `self.danio // 2`.
3. Metelos a los cuatro en una lista y recorrela con un `for`.
4. **Sin ejecutar**, escribí primero qué esperás que salga. Después comparalo.

La reflexión: agregaste dos enemigos nuevos. ¿Tuviste que cambiar una sola línea del `for`? ¿Qué pasaría si lo hubieras hecho con `if` y `elif`, y mañana quisieras veinte enemigos?

09

ETAPA 9 • TODO JUNTO

Construir la pelea

OBJETIVO

Hacer chocar todo lo que construiste: herencia, composición, `return` y polimorfismo, en un solo `while`.

Todas las piezas existen y ninguna se toca. Tenés un jugador con arma, un enemigo que pega y dos personajes que saben decir si siguen vivos. **Falta el pedazo de programa que los haga pelear.**

ANTES DE PROGRAMAR

Ordená el turno

Numerá del 1 al 7 en qué orden tienen que pasar las cosas en cada vuelta del combate:

- ___ Si sigue vivo, el enemigo ataca
- ___ El jugador ataca y obtenemos el daño
- ___ Cuando uno cae, se muestra el ganador
- ___ El enemigo recibe ese daño
- ___ Se repite todo mientras los dos estén vivos
- ___ El jugador recibe el daño del enemigo
- ___ Se muestra cómo quedó la vida del enemigo

Pedazo 1 • la condición del `while`

```
while aria.esta_vivo() and goblin.esta_vivo():
```

La pelea sigue **mientras los dos estén vivos**. Va `and` y no `or` porque basta con que **uno** se muera para que la condición sea falsa y el combate termine; con `or`, el sobreviviente le seguiría pegando a un muerto.

MIRÁ BIEN ESA LÍNEA

Es **puro** `return`. Los objetos contestan `True` o `False` y el programa decide con eso. Si `esta_vivo()` hiciera `print`, esta pelea sería imposible de escribir. Ahí se cobra todo lo de la clase 2.

Pedazo 2 · el turno del jugador

```
danio = aria.atacar()
goblin.recibir_danio(danio)
print(aria.nombre, "hace", danio, "de daño .", goblin.nombre, "queda con", goblin.vida)
```

Tres líneas, tres objetos hablando entre ellos. Esto es POO funcionando.

- | | |
|---------|--|
| LÍNEA 1 | El jugador le pregunta a su arma cuánto pega. Como <code>atacar()</code> usa <code>return</code> , ese número le queda en la mano, guardado en <code>danio</code> . Si el arma hiciera <code>print</code> , acá no habría nada que guardar. |
| LÍNEA 2 | El jugador le manda ese número al otro personaje , que se lo resta a su propia vida. Fijate que el jugador no toca la vida del otro a mano: se lo pide . Cada objeto se ocupa de lo suyo. |
| LÍNEA 3 | Ahora sí un <code>print</code> : esto lo lee una persona , no lo necesita el programa. |

Pedazo 3 · el contraataque (y un detalle importante)

Si el goblin quedó con 0 de vida y el enemigo igual contraataca, **un enemigo muerto le sigue pegando al jugador**. El programa no da error: por eso es peligroso. Anda igual, pero el juego queda mal. El

`while` corta recién en la vuelta *siguiente*.

```
if goblin.esta_vivo():
    golpe = goblin.atacar()
    aria.recibir_danio(golpe)
    print(goblin.nombre, "hace", golpe, "de daño .", aria.nombre, "queda con", aria.vida)
```

De nuevo, el método que **devuelve** `True` o `False` es el que nos salva.

Todo junto

```
espada = Arma("Espada", 25)
aria = Jugador("Aria", 100, espada)
goblin = Goblin("Goblin", 60, 12)

print("¡Empieza el combate!")

while aria.esta_vivo() and goblin.esta_vivo():
    danio = aria.atacar()
    goblin.recibir_danio(danio)
    print(aria.nombre, "hace", danio, "de daño .", goblin.nombre, "queda con", goblin.vida)

    if goblin.esta_vivo():
        golpe = goblin.atacar()
        aria.recibir_danio(golpe)
        print(goblin.nombre, "hace", golpe, "de daño .", aria.nombre, "queda con", aria.vida)

if aria.esta_vivo():
    print("Ganó", aria.nombre)
else:
    print("Ganó", goblin.nombre)
```

Este código va **abajo de todas las clases**. Con estos números Aria gana en tres golpes y queda con 76 de vida: hacé la cuenta a mano antes de ejecutarlo.

¿Dónde está cada herramienta?

HERENCIA Y SUPER()

```
class Jugador(Personaje),  
class Goblin(Enemigo) y  
super().__init__(nombre, vida).  
Nadie volvió a escribir vida.
```

COMPOSICIÓN

```
self.arma = arma, y después  
self.arma.usar(): el jugador le  
pregunta a su arma.
```

RETURN

```
esta_vivo() hace funcionar al  
while; atacar() entrega el  
número que se resta.
```

POLIMORFISMO

Cambiá Goblin por Dragon en una sola línea y la pelea entera se comporta distinto. Sin tocar el while.

ENCAPSULACIÓN

El if adentro de recibir_danio(): nadie toca la vida desde afuera y nunca queda en negativo.

ABSTRACCIÓN

aria.atacar() es una línea. Adentro pasan varias cosas, pero eso no es problema de quien usa el objeto.

EN LA COMPU

Desafío de la etapa

1. Escribí la pelea completa y ejecutala.
2. Cambiá el goblin por un Dragon en una sola línea. ¿Cambiaste algo del while?
3. Agregá un contador de vueltas y mostrá al final: "El combate duró 5 turnos".
4. **La difícil.** En la clase 2 el ataque estaba empaquetado adentro del jugador:

```
def atacar_a(self, otro):  
    danio = self.arma.usar()  
    otro.recibir_danio(danio)  
    return danio
```

Escribilo en tu Jugador y reemplazá las dos líneas del turno por aria.atacar_a(goblin). Después contestá: **si ya le restó la vida al otro, ¿para qué encima devuelve el daño?**

ETAPAS 10 Y 11

10

Experiencia, niveles y menú

ETAPA 10 • OBJETIVO

Agregarle al jugador información nueva (nivel, experiencia) y el comportamiento que la maneja.

Ganaste la pelea... y no pasó absolutamente nada. Tu personaje sigue igual de fuerte que al empezar. **Falta la recompensa.**

Fijate que cuando un programa crece siempre es lo mismo: o le agregamos **datos** a un objeto, o le agregamos **acciones**. Acá agregamos las dos cosas. Y van en **Jugador**, no en **Personaje**: solo el jugador sube de nivel.

```
class Personaje:  
    def __init__(self, nombre, vida):  
        self.nombre = nombre  
        self.vida = vida  
        self.vida_maxima = vida # ← nuevo: hasta dónde se puede curar
```

Esta sí va arriba: cualquier personaje puede tener un techo de vida.

```
class Jugador(Personaje):
    def __init__(self, nombre, vida, arma):
        super().__init__(nombre, vida)
        self.arma = arma
        self.nivel = 1          # ← nuevo
        self.experiencia = 0    # ← nuevo

    def ganar_experiencia(self, cantidad):
        self.experiencia = self.experiencia + cantidad
        print(self.nombre, "ganó", cantidad, "de experiencia")
        if self.experiencia ≥ 100:
            self.subir_nivel()

    def subir_nivel(self):
        self.nivel = self.nivel + 1
        self.experiencia = self.experiencia - 100
        self.vida_maxima = self.vida_maxima + 20
        self.vida = self.vida_maxima
        print(self.nombre, "subió al nivel", self.nivel)
```

`nivel` y `experiencia` **no se piden por parámetro**: todos los jugadores arrancan en nivel 1 con 0. Un atributo puede tener un valor fijo de arranque, sin que nadie se lo pase — la misma idea del bonus de la clase 2: *si todas las motos tienen 2 ruedas, no hace falta pedir el dato*.

¿POR QUÉ RESTAR 100 Y NO PONER 0?

Si ganó 130 de experiencia, sube de nivel y le **sobran 30**. Con `= 0` esos 30 se pierden; restando, quedan para el próximo nivel. Es una decisión de diseño: la otra también es válida, si la podés justificar.

¿QUIÉN LLAMA A `SUBIR_NIVEL()` ?

El propio jugador, desde adentro de `ganar_experiencia()`. Vos no tenés que acordarte de nada: **el objeto se encarga solo**. Eso es encapsulación otra vez.

PREDICCIÓN **Hacé la cuenta a mano**

```
aria.ganar_experiencia(60)
aria.ganar_experiencia(60)
print(aria.nivel)
print(aria.experiencia)
```

¿Qué imprimen las dos últimas líneas? **Y una pregunta más**: ¿por qué usamos `≥ 100` y no `= 100` ?

EN LA COMPU **Desafíos de la etapa**

- Hacé que al ganar la pelea el jugador reciba experiencia: `aria.ganar_experiencia(50)`.
- Pensalo antes de escribirlo**: ¿qué más podría mejorar cuando el jugador sube de nivel? Elegí una mejora y programala.
- La difícil**: ¿cómo harías para que el arma pegue más fuerte al subir de nivel? Hay dos caminos: tocarle el daño al arma (`self.arma.danio = self.arma.danio + 5`) o darle a `Arma` un método `mejorar()` y pedírselo. **¿Cuál te parece mejor y por qué?** Esa pregunta vale más que el código.
- Que el enemigo dé más experiencia si es más difícil.

⚠ QUE QUEDE CLARO

El menú no es el objetivo del trabajo práctico. Es solamente la forma de *usar* el sistema que construiste. Lo importante ya está hecho: las clases, la herencia, la composición, el `return` y el polimorfismo. El menú son `if` y `elif`, y eso ya lo sabías hacer desde antes de POO.

Igual, sin menú el juego no se juega: se ejecuta y se termina solo. Y antes falta un objeto, porque una de las opciones va a ser curarse. ¿Un jugador ES una poción? No. ¿TIENE una? Sí: es exactamente el mismo caso que el arma.

```
class Pocion:
    def __init__(self, nombre, cura):
        self.nombre = nombre
        self.cura = cura

    def usar(self):
        return self.cura
```

```
def curarse(self, pocion):
    self.vida = self.vida + pocion.usar()
    if self.vida > self.vida_maxima:
        self.vida = self.vida_maxima
```

Arma y Pocion son casi idénticas: guardan un número y lo devuelven. Cuando entendés un patrón, el siguiente objeto sale solo.

El jugador tampoco calcula nada: le pregunta a la poción cuánto cura. Y ese `if` es el mismo control que en `recibir_danio()`, pero del otro lado: uno impide bajar de 0, el otro impide pasar del máximo.

```
while True:
    print("=== ARENA DE COMBATE ===")
    print("1. Ver información")
    print("2. Atacar")
    print("3. Usar poción")
    print("4. Ver experiencia")
    print("5. Salir")
    opcion = input("Elegí una opción: ")

    if opcion == "1":
        heroe.mostrar_info()
    elif opcion == "5":
        print("¡Hasta la próxima!")
        break
```

Dos detalles de Python: `input()` siempre devuelve **texto**, por eso comparamos con `"1"` entre comillas y no con `1`. Y `while True` repite para siempre: la única forma de salir es el `break`. El programa completo está en la sección siguiente.

11

PARA COMPARAR CON EL TUYO

El programa completo

Todo lo que construiste en once etapas, junto.

Leelo entero antes de copiarlo: tenés que poder explicar cada línea. Si hay alguna que no entendés, volvé a la sección donde apareció. Y no lo uses como punto de partida: el trabajo es llegar hasta acá, no arrancar desde acá.

```
# ===== ARENA DE COMBATE =====  
# Trabajo práctico integrador de P00
```

```
class Personaje:  
    def __init__(self, nombre, vida):  
        self.nombre = nombre  
        self.vida = vida  
        self.vida_maxima = vida  
  
    def recibir_danio(self, cantidad):  
        self.vida = self.vida - cantidad  
        if self.vida < 0:  
            self.vida = 0  
  
    def esta_vivo(self):  
        return self.vida > 0  
  
    def mostrar_info(self):  
        print(self.nombre, "- Vida:", self.vida, "/", self.vida_maxima)
```

```
class Arma:  
    def __init__(self, nombre, danio):  
        self.nombre = nombre  
        self.danio = danio  
  
    def usar(self):  
        return self.danio
```

```
class Pocion:  
    def __init__(self, nombre, cura):  
        self.nombre = nombre  
        self.cura = cura  
  
    def usar(self):  
        return self.cura
```

```
class Jugador(Personaje):  
    def __init__(self, nombre, vida, arma):  
        super().__init__(nombre, vida)  
        self.arma = arma  
        self.nivel = 1  
        self.experiencia = 0  
  
    def atacar(self):  
        return self.arma.usar()  
  
    def curarse(self, pocion):  
        self.vida = self.vida + pocion.usar()  
        if self.vida > self.vida_maxima:  
            self.vida = self.vida_maxima  
  
    def ganar_experiencia(self, cantidad):  
        self.experiencia = self.experiencia + cantidad  
        print(self.nombre, "ganó", cantidad, "de experiencia")  
        if self.experiencia ≥ 100:  
            self.subir_nivel()  
  
    def subir_nivel(self):  
        self.nivel = self.nivel + 1  
        self.experiencia = self.experiencia - 100
```

```

        self.vida_maxima = self.vida_maxima + 20
        self.vida = self.vida_maxima
        print(self.nombre, "subió al nivel", self.nivel)

class Enemigo(Personaje):
    def __init__(self, nombre, vida, danio):
        super().__init__(nombre, vida)
        self.danio = danio

    def atacar(self):
        return self.danio

class Goblin(Enemigo):
    def atacar(self):
        return self.danio          # igual que el padre: podría no escribirse

class Dragon(Enemigo):
    def atacar(self):
        return self.danio + 15     # el dragón pega más fuerte

# ===== creamos los objetos =====
espada = Arma("Espada", 25)
pocion = Pocion("Poción chica", 30)
heroe = Jugador("Aria", 150, espada)    # 150 de vida: el dragón pega fuerte
enemigo = Dragon("Dragón rojo", 90, 20) # 20 + 15 = 35 de daño por golpe

# ===== el menú =====
while True:
    print()
    print("=== ARENA DE COMBATE ===")
    print("1. Ver información")
    print("2. Atacar")
    print("3. Usar poción")
    print("4. Ver experiencia")
    print("5. Salir")
    opcion = input("Elegí una opción: ")

    if opcion == "1":
        heroe.mostrar_info()
        enemigo.mostrar_info()

    elif opcion == "2":
        danio = heroe.atacar()
        enemigo.recibir_danio(danio)
        print(heroe.nombre, "atacó con", heroe.arma.nombre, "e hizo", danio, "de daño")

        if enemigo.esta_vivo():
            golpe = enemigo.atacar()
            heroe.recibir_danio(golpe)
            print(enemigo.nombre, "contraatacó e hizo", golpe, "de daño")
        else:
            print(enemigo.nombre, "fue derrotado")
            heroe.ganar_experiencia(50)

    elif opcion == "3":
        heroe.curarse(pocion)
        print(heroe.nombre, "usó", pocion.nombre, "- Vida:", heroe.vida)

    elif opcion == "4":
        print("Nivel:", heroe.nivel, "- Experiencia:", heroe.experiencia, "/ 100")

```

```

elif opcion == "5":
    print("¡Hasta la próxima!")
    break

else:
    print("Esa opción no existe")

if not heroe.esta_vivo():
    print("GAME OVER")
    break

```

Con estos números el héroe derrota al dragón en cuatro ataques y gana 50 de experiencia. Probá cambiar `Dragon` por `Goblin` en la línea del enemigo y contá **cuántas líneas más del programa tuviste que tocar**.

🔍 BUSCÁ LOS CONCEPTOS EN TU PROPIO CÓDIGO

Antes de seguir, marcá en tu archivo con un comentario dónde está cada cosa: **herencia**, `super()`, **composición**, **polimorfismo**, `return` y **encapsulación**. Si alguna no la encontrás, esa es la que tenés que repasar. Y además te va a servir para la entrega.

12

ETAPA 12

Desafíos de ampliación

Están ordenados de menor a mayor dificultad. **No hace falta hacerlos todos.**

Vale mucho más hacer dos bien y poder explicarlos, que hacer cinco a medias. Y antes de empezar cualquiera, hacé lo de siempre aunque sea en dos líneas: **¿qué objetos aparecen?** **¿qué información tienen?** **¿qué pueden hacer?** **¿cómo se relacionan?**

NIVEL 1

Poblar la arena

- ☐ Creá **otra arma** con distinto daño y hacé que el jugador pueda elegir con cuál pelear.
- ☐ Creá **otro enemigo** con distinta vida y daño.
- ☐ Agregale un método `mostrar_info()` a `Arma` y otro a `Enemigo`.

Qué practicás: clase vs objeto · atributos · métodos · `print()` vs `return`.

La pregunta: `mostrar_info()`, ¿usa `print()` o `return`? ¿Por qué?

NIVEL 2**Pociones que curan de verdad**

- ☐ Si te saltaste la etapa 11: creá la clase **Pocion** y el método `curarse()`.
- ☐ Hacé que la vida **nunca supere** la `vida_maxima`. Probalo curándote con la vida casi llena.
- ☐ Que la poción tenga **usos limitados**: un atributo `usos` que baje cada vez y avise cuando se acabó.
- ☐ Creá una **poción grande** y una **chica**, con distinta cura.

Qué practicás: composición (TIENE UN) · encapsulación · un objeto le pregunta a otro.

La pregunta: el control de la vida máxima, ¿va adentro de `curarse()` o afuera, en el menú? ¿Quién tiene que decidir cómo cambia la vida del jugador?

NIVEL 3**Clases de personaje y habilidades**

- ☐ Creá **Guerrero**, **Mago** y **Arquero**, hijos de `Jugador`, cada uno con un atributo propio.
- ☐ Que cada uno tenga su propio `atacar()`: el guerrero suma su fuerza, el mago su magia, el arquero pega doble cada dos turnos.
- ☐ Metelos a los tres en una **lista** y recorrela con un `for`. Sin un solo `if`.
- ☐ Agregá una **habilidad especial** que se desbloquee al llegar a nivel 3.

Qué practicás: herencia de varios niveles · `super()` · polimorfismo.

La pregunta: cuando agregaste el tercer personaje al `for`, ¿tuviste que cambiar el `for`? Esa es exactamente la ventaja del polimorfismo.

NIVEL 4**Inventario**

- ☐ Creá una clase **Inventario** que guarde una **lista** de objetos adentro.
- ☐ Métodos `agregar()`, `mostrar()` y `cantidad()`. Decidí cuál usa `return`.
- ☐ Que el jugador **tenga** un inventario y pueda **cambiar de arma** eligiendo una de la lista.
- ☐ Agregá la opción **Ver inventario** al menú.

Qué practicás: composición con listas · objetos adentro de objetos · `return`.

La pregunta: el jugador tiene un inventario, y el inventario tiene armas. ¿Cómo se lee `jugador.inventario.armas[0].nombre`?

- ☐ Agregale **monedas** al jugador y hacé que ganar una pelea dé monedas.
- ☐ Creá una clase **Tienda** con un método `comprar()` que revise si al jugador le alcanza. **Ese método tiene que devolver algo.**
- ☐ Creá una **lista de enemigos** y peleá contra todos, uno tras otro, con un `for`.
- ☐ Que el último sea un **JefeFinal** (hijo de `Enemigo`) con un ataque especial.
- ☐ Entre pelea y pelea, que el jugador pueda curarse o ir a la tienda.

Qué practicás: todo junto · varios objetos colaborando · herencia + composición + polimorfismo · diseño.

La pregunta: el `for` que recorre los enemigos, ¿necesita saber si el que viene es un goblin, un dragón o el jefe final?

⚠ CUIDADO CON UNA TRAMPA

No agregues clases **porque sí**. Si creás `Moneda`, `PuntoDeVida` y `NumeroDeTurno` como clases, volviste al desorden del principio. Preguntate siempre: **¿esto tiene información propia y hace algo?** Si no, es un atributo de alguien.

Cuando algo no anda

Estos errores los vas a cometer sí o sí. Lo importante es **reconocer el síntoma** para saber enseguida qué te está pasando.



Olvidarte el `super()`

`AttributeError: ... has no attribute 'nombre'`

El padre nunca hizo su parte, así que ese atributo no existe. Andá al `__init__` del hijo y fijate si está la llamada al padre, como **primera línea**.



Usar `print` donde iba `return`

Aparece `None` donde esperabas un número

El método mostró el valor pero no lo entregó. Cambiá `print` por `return` y volvé a probar. Si el `while` de la pelea no corta nunca, casi seguro es esto.



Heredar cuando había que componer

"un jugador es un arma" 😞

Si la frase "es un" suena rara al leerla en voz alta, no era herencia: era **tiene un**. Sacá el `(Arma)` y poné `self.arma = arma`.



Olvidarte el `self.`

El dato "desaparece" después del `__init__`

Si escribiste `vida = vida` en vez de `self.vida = vida`, el dato no se guardó en el objeto: quedó en una variable que murió ahí. Es el error número uno de la clase 1.



Comparar texto con número

El menú no entra en ninguna opción

`input()` devuelve **texto**. Hay que comparar con `opcion == "1"`, entre comillas, y no con `opcion == 1`.



Indentación

`IndentationError` · el método quedó afuera

Los métodos van indentados **adentro** de la clase. Si un `def` quedó pegado al margen, dejó de ser un método y pasó a ser una función suelta.

Preguntas que te sacan de casi cualquier atolladero

SI NO SABÉS DÓNDE PONER ALGO

¿Esto lo tienen todos o lo tiene uno solo? Si lo tienen todos, va en la clase de arriba. Si lo tiene uno solo, va en el hijo.

SI NO SABÉS SI VA HERENCIA O COMPOSICIÓN

Leé las dos frases en voz alta: "un X es un Y" y "un X tiene un Y". La que suena natural es la correcta.

SI NO SABÉS SI VA PRINT O RETURN

¿Lo necesita una persona o lo necesita el programa?

Persona → `print()`. Programa → `return`.

SI UNA LÍNEA CON MUCHOS PUNTOS TE MAREA

Leela **de derecha a izquierda** poniendo un "de" en cada punto: `jugador.arma.nombre` es "el nombre del arma del jugador".

🕒 Y SI ESTÁS TRABADO DE VERDAD, VOLVÉ A LAS 7 PREGUNTAS DEL DISEÑO

1) ¿Qué objetos hay acá? · 2) ¿Qué debería tener la clase de arriba? · 3) ¿Qué agrega cada hijo? · 4) ¿Dónde usamos `super()`? · 5) ¿Qué objeto tiene a otro adentro? · 6) ¿Qué método debería devolver un valor? · 7) ¿Dónde herencia y dónde composición?

14

LA CONSIGNA FORMAL

Entrega y evaluación

Cada estudiante o pareja (máximo dos personas) entrega estas seis cosas.

- ☐ **1· El código completo.** El archivo `juego.py`, que tiene que **ejecutarse sin errores**. Probalo de cero antes de entregar.
- ☐ **2· Una explicación del diseño.** Media carilla: qué objetos identificaste, qué decisiones tomaste y **por qué**. Las decisiones discutibles, mejor: explicá por qué elegiste esa.
- ☐ **3· El diagrama de clases y relaciones.** Cajas y flechas, como los de la clase 2, con las flechas de **ES UN** y **TIENE UN** bien marcadas. Puede ser a mano.
- ☐ **4· Una demostración del juego funcionando.** Una pelea completa, una subida de nivel y el menú andando.
- ☐ **5· Dónde usaste cada concepto de POO.** Una lista con el número de línea: herencia, `super()`, composición, polimorfismo, `return` y encapsulación.
- ☐ **6· Una lista de mejoras posibles.** Tres cosas que le agregarías si tuvieras más tiempo, y **qué clase tocarías** para hacer cada una.

⚠️ SI TRABAJARON EN PAREJA

Los dos tienen que poder explicar **todo** el código, no solo la parte que escribieron. En la defensa se le pregunta a cualquiera de los dos, sobre cualquier parte.

Cómo se evalúa

Se evalúa principalmente **el razonamiento y la comprensión**, no la cantidad de funcionalidades. Un juego chiquito que entendés a fondo vale más que uno enorme que no sabés explicar.

QUÉ SE MIRA	LO ESPERADO
Identificación de objetos	Encontró los objetos del problema y no convirtió en clase cualquier cosa que se le ocurrió.
Diseño de clases	Lo común está arriba, lo específico abajo. Cada clase tiene un motivo para existir.
Atributos y métodos	Los atributos son datos y están en la clase correcta; los métodos son acciones y hacen una cosa cada uno.
Herencia	Se usa donde se puede decir "ES UN", y no para ahorrar líneas porque sí.
Uso de <code>super()</code>	El hijo no repite los atributos del padre. Puede explicar qué pasa si lo saca.
Composición	Hay al menos un objeto adentro de otro, y sabe justificar por qué ahí no iba herencia.
Polimorfismo	Distintos objetos responden al mismo método a su manera, y el código que los usa no tiene cadenas de <code>if</code> .
Uso de <code>return</code>	Devuelve cuando el programa necesita el valor; imprime cuando lo lee una persona. Puede explicar cada caso.
Funcionamiento	El programa corre sin errores y hace lo que dice que hace.
Capacidad de explicar las decisiones	Lo que más pesa. Puede contar por qué diseñó así y qué otra opción había.
Prolijidad y mejoras	Nombres claros, código ordenado, y una idea concreta de cómo seguiría.

💡 LO QUE NADIE TE VA A PEDIR

Que recites la definición de polimorfismo. Lo que importa es que, frente a un problema, **sepas encontrar los objetos y hacerles las dos preguntas**. El resto es vocabulario y se aprende usándolo.

15

ANTES DE ENTREGAR

Autoevaluación

Primera parte: ¿lo podés explicar?

Marcá cada casilla **solo si podés explicarlo en voz alta, sin leer**. Si alguna te queda sin marcar, volvé a esa sección: ahí está lo que te falta.

- ☐ Qué **clases creé** y por qué existe cada una.
- ☐ Qué **objetos creé** a partir de esas clases, y en qué se diferencia una clase de un objeto.
- ☐ Qué **atributos usé** y por qué cada uno está en *esa* clase y no en otra.
- ☐ Qué **métodos implementé** y qué hace cada uno.
- ☐ Dónde **apliqué herencia** y por qué ahí sí se podía decir "ES UN".
- ☐ Dónde **usé** `super()` y qué pasaría si lo saco.
- ☐ Dónde **aparece la composición** y por qué ahí no servía la herencia.
- ☐ Dónde **aparece el polimorfismo** y qué problema me ahorró.
- ☐ Qué **métodos usan** `return` y por qué esos y no otros.
- ☐ Qué **problema resolvió** la POO acá: qué hubiera pasado sin objetos.
- ☐ Qué **mejoraría** de mi proyecto si tuviera una semana más.

Segunda parte: ejercicios de repaso

1 ES UN o TIENE UN

Para cada par, decidí si va herencia o composición. Leelos en voz alta.

- Una **Mochila** y un **Libro**
- Un **JefeFinal** y un **Enemigo**
- Una **Tienda** y una **Moneda**
- Un **Mago** y un **Jugador**
- Un **Inventario** y un **Arma**

2 ¿Dónde va este atributo?

Escribí en qué clase de *nuestro* juego tiene que estar cada uno, y por qué: `nombre` · `vida` · `experiencia` · `arma` · `danio` (el de la espada) · `danio` (el del goblin) · `vida_maxima` .

3

Predecí la salida

```
espada = Arma("Espada", 25)
aria = Jugador("Aria", 100, espada)
goblin = Goblin("Goblin", 60, 12)

goblin.recibir_danio(aria.atacar())
print(goblin.vida)
goblin.recibir_danio(aria.atacar())
goblin.recibir_danio(aria.atacar())
print(goblin.vida)
print(goblin.esta_vivo())
```

Escribí qué imprime cada `print`. Hacé las cuentas a mano.

4

Encontrá los tres errores

```
class Enemigo(Personaje):
    def __init__(self, nombre, vida, danio):
        danio = danio

    def atacar(self):
        print(self.danio)

goblin = Enemigo("Goblin", 60, 12)
print(goblin.nombre)
danio = goblin.atacar()
print(danio + 5)
```

Hay tres cosas mal. Encontralas, decí qué error da cada una y arreglalas.

5

Diseñá otra cosa

Sin escribir código: diseñá el sistema de una **veterinaria** usando las 7 preguntas del diseño. Entregá un dibujo con cajas y flechas. Tienen que aparecer al menos una **herencia**, una **composición** y un método que **devuelva** un valor.

Este ejercicio parece que no tiene nada que ver con el trabajo práctico. **Es el más importante de todos:** si te sale, entendiste que POO no era sobre videojuegos.

6

La reflexión final

Si mañana te damos un problema completamente diferente —una veterinaria, un cine, una app de música—, **¿qué deberías preguntarte antes de escribir código?** Escríbilo con tus palabras.

Están acá al final **a propósito**: si las tenés al lado del enunciado, el ojo las lee antes de que la cabeza intente. Resolvé primero, aunque te salga mal — equivocarte y después entender por qué es la parte que te lo deja aprendido.

En varios ejercicios **no hay una única respuesta correcta**. Si la tuya es distinta pero pasa el test de leerla en voz alta, está bien.

Sección 3 · Actividad 1 · Los objetos de la escena

Sí son objetos: el héroe, el goblin, la espada y la poción. Los cuatro guardan información y hacen algo.

No son objetos: la **barra de vida** (es la forma de *mostrar* un dato; la vida es un atributo del personaje), los **40 puntos de experiencia** (un número que guarda el jugador: atributo suyo) y el **fondo** (no tiene información ni hace nada: es decoración).

Sección 3 · Actividad 2 · Atributo o método

Atributos: `nombre`, `vida`, `experiencia` — son sustantivos, cosas que el héroe *tiene*.

Métodos: `atacar()`, `recibir_danio()`, `subir_nivel()` — son verbos con paréntesis, cosas que el héroe *hace*.

Sección 3 · Actividad 3 · La tabla de diseño

CLASE	ATRIBUTOS	MÉTODOS
Personaje	nombre, vida	recibir_danio(), esta_vivo()
Jugador	nivel, experiencia, arma	atacar(), ganar_experiencia()
Enemigo	danio	atacar()
Arma	nombre, danio	usar()

En Jugador y en Enemigo **no se repiten** `nombre` ni `vida`: eso ya está en Personaje y lo heredan. Si agregaste `vida_maxima` o `mostrar_info()`, también está bien.

Sección 4 · Predicciones y el arreglo de `recibir_danio()`

1. Quedan **70**. 2. Devuelve **False**: la cuenta `0 > 0` da falso. 3. La vida queda en **-400**, y por eso hay que arreglarlo:

```
def recibir_danio(self, cantidad):  
    self.vida = self.vida - cantidad  
    if self.vida < 0:  
        self.vida = 0    # nunca menos de 0
```

Ese `if` es la única razón por la que la vida no queda en negativo. Si lo sacás, el pilar de la **encapsulación** desaparece.

Sección 5 · Las armas y ¿print o return?

`espada.usar()` → 25 · `arco.usar()` → 15 · `espada.nombre` → **Espada** · `resultado - 5` → 20.

¿print o return? 1. `return` (el daño se lo restamos a la vida del enemigo). 2. `print()` (lo lee una persona). 3. `return` (la respuesta va adentro de un `if`). 4. `print()` (nadie lo reutiliza). 5. `return` (con ese número después decidís si sube de nivel).

Sección 6 · El error del `super()` y ES UN / TIENE UN

El error: `AttributeError: 'Jugador' object has no attribute 'nombre'`. Falta `super().__init__(nombre, vida)` como primera línea: **el padre nunca hizo su parte**, así que `nombre` jamás se guardó. Heredar la clase no alcanza: hay que llamar al `__init__` del padre.

Los pares: 1. ES UN · 2. TIENE UN · 3. TIENE UN · 4. ES UN · 5. TIENE UN · 6. ES UN · 7. TIENE UN · 8. ES UN.

Sección 7 · El objeto goblin y el Guerrero

Funcionan: `goblin.nombre` y `goblin.vida` (los puso `Personaje` a través del `super()`), `goblin.danio` (propio de `Enemigo`), y `goblin.esta_vivo()` y `goblin.recibir_danio(20)` — aunque **no estén escritos adentro de `Enemigo`**: los heredó.

Da error: `goblin.arma`. El arma es del `Jugador`, no de `Personaje`.

El Guerrero: imprime **Aria, Espada, 8 y True**. La que falla es `aria.danio`: ninguna clase de la cadena guarda "danio" en el jugador — el daño lo tiene el arma. Sería `aria.arma.danio`.

Sección 8 · El `for` polimórfico

Imprime **Dragón 35** (20 + 15), **Goblin 12** y **Lobo 10**. El `+15` es solo del dragón: está escrito adentro de `Dragon`, los goblins no lo heredan.

Y hay **cero `if`** en ese `for`: cada objeto sabe cómo responder.

Sección 9 · El orden del turno

1. El jugador ataca y obtenemos el daño · **2.** El enemigo recibe ese daño · **3.** Se muestra cómo quedó la vida del enemigo · **4.** Si sigue vivo, el enemigo ataca · **5.** El jugador recibe el daño del enemigo · **6.** Se repite mientras los dos estén vivos · **7.** Cuando uno cae, se muestra el ganador.

Y la difícil: `atacar_a()` devuelve el daño aunque ya se lo haya restado al otro porque **el golpe ya está dado, pero el programa todavía no sabe de cuánto fue**. Con el `return` podés mostrarlo, sumarlo a un total o guardar el récord del golpe más fuerte. Sin él, esa información se pierde.

Sección 10 · Experiencia

Imprime **nivel 2** y **experiencia 20**. La cuenta: 60 + 60 = 120; como `120 ≥ 100`, sube de nivel y le restamos 100, así que le quedan 20.

Usamos `≥` y no `=` porque con `=` solo subiría si cayera **justo** en 100. Con 120 no subiría nunca.

Sección 15 · Ejercicios de repaso

1. 1) TIENE UN · 2) ES UN · 3) TIENE UN · 4) ES UN · 5) TIENE UN.

2. `nombre`, `vida` y `vida_maxima` → **Personaje** (los comparten todos). `experiencia` y `arma` → **Jugador** (solo él sube de nivel y solo él lleva arma). `danio` de la espada → **Arma**. `danio` del goblin → **Enemigo**.

3. Primer `print`: **35** (60 - 25). Segundo: **0** — ojo, no -15, porque el `if` de `recibir_danio()` lo frena en 0. Tercero: **False**.

4. Los tres errores: (a) falta `super().__init__(nombre, vida)`, así que `goblin.nombre` da `AttributeError`; (b) dice `danio = danio` en vez de `self.danio = danio`, así que el dato no se guarda; (c) `atacar()` hace `print` en vez de `return`, así que `danio` vale `None` y `danio + 5` revienta.

5. Una salida posible: **herencia** — `Perro` y `Gato` son ambos `Animal`. **Composición** — un `Turno` *tiene* un `Animal` y un `Veterinario`; una `HistoriaClinica` *tiene* una lista de `Consulta`. **Return** — `turno.precio()` devuelve cuánto hay que cobrar. Y algo que **no** va: `Consultorio` no hereda de `Veterinaria` — el consultorio está *adentro*, no es una veterinaria.

6. No hay una única forma de decirlo, pero tienen que estar estas preguntas, en este orden: **¿Qué problema tengo?** → **¿Qué objetos existen?** → **¿Qué información tiene cada uno?** → **¿Qué puede hacer?** → **¿Cómo se relacionan?** Y recién después, el código. Si tu respuesta empieza por "abro Python y...", volvé a leerla. Fijate que en ninguna de esas preguntas aparece la palabra Python: por eso el método te sirve para cualquier problema y en cualquier lenguaje.

PARA CERRAR

No estamos aprendiendo comandos sueltos. Estamos aprendiendo a pensar, diseñar y construir programas.

Empezaste con una pantalla en blanco y terminaste con un videojuego funcionando. Pero lo que de verdad te llevás no es el juego: es que ahora, frente a cualquier problema, sabés por dónde empezar.

Y la primera pregunta va a seguir siendo siempre la misma: **"¿qué objetos hay acá?"** Y ahora, además, una segunda: **"¿y cómo se relacionan entre ellos?"**

 LAS DOS PREGUNTAS DE SIEMPRE

¿Qué información tiene? → atributos (sustantivos).

¿Qué puede hacer? → métodos (verbos con paréntesis).

Si dudás, decilo en voz alta: "el personaje tiene..." / "el personaje puede..."

 SI PODÉS DECIR "TIENE UN"

Composición

```
self.arma = arma
```

"Un jugador **tiene un arma**." Dos objetos distintos, conectados.

 PRINT() VS RETURN

¿Lo necesita una persona o el programa?

`print()` → a la pantalla. Lo ve alguien y ahí termina.

`return` → me lo entrega. Lo guardo, lo comparo en un `if`, lo reutilizo.

 EL PUNTO SE LEE "DE"

```
jugador.arma.nombre
```

= "el nombre **del** arma **del** jugador". De derecha a izquierda.

Un atributo —y también un parámetro— puede ser otro objeto entero.

 SI PODÉS DECIR "ES UN"

Herencia

```
class Jugador(Personaje):
```

"Un jugador **es un** personaje." Una sola cosa que recibe todo lo del padre.

 SUPER()

```
super().__init__(nombre, vida)
```

= "Padre, encargate vos de nombre y vida. Yo me ocupo del resto."

Va como **primera línea** del `__init__` del hijo.

 POLIMORFISMO

Misma orden, distinta respuesta. Cada enemigo tiene su `atacar()`.

El `for` que los recorre **no tiene un solo if**, y no cambia nunca aunque agregues veinte enemigos.

 LOS ERRORES · SÍNTOMA → CAUSA

- 1 `AttributeError` → falta el `super()`.
- 2 Aparece `None` → `print` donde iba `return`.
- 3 "Es un" suena raro → era composición.
- 4 El dato desaparece → te olvidaste el `self`.

 LAS CLASES DEL JUEGO

- 1 **Personaje** · nombre, vida · recibir_danio(), esta_vivo()
- 2 **Arma** · nombre, danio · usar()
- 3 **Jugador**(Personaje) · arma, nivel, experiencia · atacar()
- 4 **Enemigo**(Personaje) · danio · atacar()

 LAS 7 PREGUNTAS DEL DISEÑO

- 1 ¿Qué objetos hay acá?
- 2 ¿Qué debería tener la clase de arriba?
- 3 ¿Qué agrega cada hijo?
- 4 ¿Dónde usamos `super()`?
- 5 ¿Qué objeto tiene a otro adentro?
- 6 ¿Qué método debería devolver un valor?
- 7 ¿Dónde herencia y dónde composición?

 Y EL ORDEN, QUE NO CAMBIA NUNCA

1 · Pensar el problema → 2 · Identificar los objetos → 3 · Atributos y métodos →

4 · Pensar las relaciones → 5 · Diseñar → 6 · Código → 7 · Probar → 8 · Mejorar

El código está **sexto**. Cinco pasos antes de tocar el teclado. **No memorices: razoná.**